

LAPORAN SIMULASI MUTUAL EXCLUSION (MUTEX)



**DIPERSIAPKAN OLEH:
SISTEM TERDISTRIBUSI**

**BRADLEY WIDJAJA
5025231036**

Teknik Informatika
Fakultas Teknik Elektro dan Informatika Cerdas
Institut Teknologi Sepuluh Nopember
2025

Pendahuluan

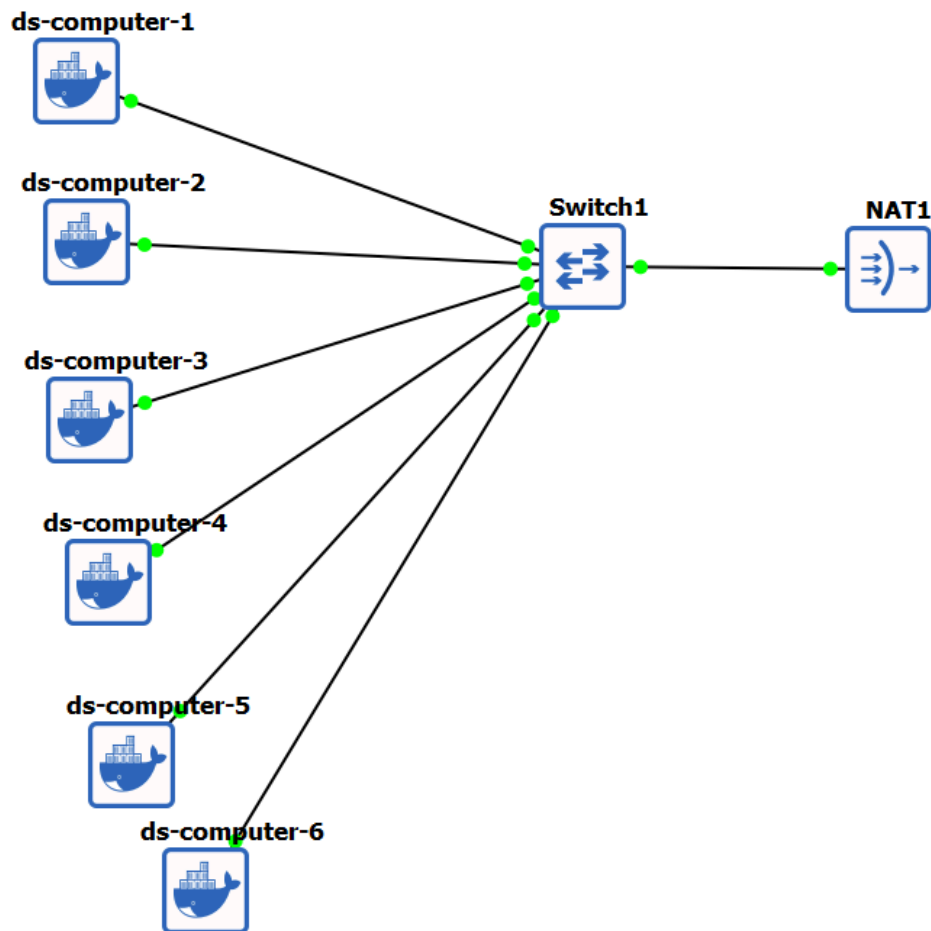
Pada suatu sistem terdistribusi, beberapa proses dapat berjalan dalam suatu waktu yang bersamaan. Hal tersebut akhirnya menyebabkan adanya beberapa akses terhadap suatu shared resource di dalam waktu yang sama. Tanpa menggunakan mutual exclusion, hal tersebut dapat menyebabkan berbagai masalah seperti munculnya race condition, inkonsistensi data, dan kegagalan dari sistem. Oleh karena itu, tugas ini bertujuan untuk melakukan simulasi terkait algoritma mutual exclusion menggunakan environment GNS3 dan beberapa node yang berkomunikasi menggunakan protokol jaringan.

Dalam tugas ini, dilakukan simulasi mutual exclusion menggunakan GNS3 dengan setiap komponen sistem berjalan secara terpisah di beberapa node dan berkomunikasi menggunakan protokol tertentu. Eksperimen dijalankan dengan 4 node berupa node_1, node_2, node_3, dan 1 logger untuk memantau aktivitas yang dilakukan antara node - node tersebut.

Tugas yang Diberikan

1. Menjalankan simulasi mutual exclusion (mutex) dari repository git:
<https://github.com/rm77/ds25/tree/main/synchronization/mutex>
2. Menjalankan komponen logger, node1, node2, dan node3 menggunakan GNS3.
3. Mendokumentasikan cara menjalankan eksperimen disertai dengan screenshot.
4. Menjelaskan arsitektur percobaan, protokol komunikasi, dan mekanisme mutual exclusion.

Topologi



Penjelasan:

Topologi terdiri dari 6 hostname yang berbasis docker container, yaitu ds-computer-1 sebagai node 1, ds-computer-2 sebagai node 2, ds-computer-3 sebagai node 3, ds-computer-4 sebagai logger, ds-computer-5 sebagai node 4, dan ds-computer-6 sebagai node-5. Keenam node tersebut terhubung ke Switch1. Node logger pada hostname ds-computer-4 berfungsi untuk menerima, menyimpan, dan melakukan analisis message terkait operasi mutex dari node 1, node 2, node 3, node 4, dan node 5 termasuk informasi tentang lock request, lock acquisition, dan lock release. Dengan log ini, dapat dilakukan analisis dan evaluasi terhadap mekanisme mutual exclusion, mendeteksi kondisi race terhadap akses shared resource ketika mengaktifkan metode mutex dengan ketika tidak menggunakan mutex, dan juga untuk mengidentifikasi bagaimana concurrency access terhadap shared resource mempengaruhi alur eksekusi dalam sistem.

Spesifikasi

Hostname	Node	IP	Port TCP	Port UDP	Spesifikasi
ds-computer-1	Node 1	192.168.122.10	8001	8101	x86_64 • Intel® Core™ i5-1235U • 1 Core / 1 Thread •

					2 GiB RAM • 20–30 GB Storage • Debian (GNS3 VM) • KVM/VMware
ds-computer-2	Node 2	192.168.122.11	8002	8102	x86_64 • Intel® Core™ i5-1235U • 1 Core / 1 Thread • 2 GiB RAM • 20–30 GB Storage • Debian (GNS3 VM) • KVM/VMware
ds-computer-3	Node 3	192.168.122.12	8003	8103	x86_64 • Intel® Core™ i5-1235U • 1 Core / 1 Thread • 2 GiB RAM • 20–30 GB Storage • Debian (GNS3 VM) • KVM/VMware
ds-computer-4	Logger	192.168.122.13	9000		x86_64 • Intel® Core™ i5-1235U • 1 Core / 1 Thread • 2 GiB RAM • 20–30 GB Storage • Debian (GNS3 VM) • KVM/VMware
ds-computer-5	Node 4	192.168.122.14	8004	8104	x86_64 • Intel® Core™ i5-1235U • 1 Core / 1 Thread • 2 GiB RAM • 20–30 GB Storage • Debian (GNS3 VM) • KVM/VMware
ds-computer-6	Node 5	192.168.122.15	8005	8105	x86_64 • Intel® Core™ i5-1235U • 1 Core / 1 Thread • 2 GiB RAM • 20–30 GB Storage • Debian (GNS3 VM) • KVM/VMware

Repository

<https://github.com/bwbrad05/Tugas-2-Sistem-Terdistribusi.git>

Penjelasan Setiap Penggunaan

1. Setup Topologi pada GNS 3
 - Membuat topologi jaringan pada GNS3 menggunakan 5 node berbasis docker container yang terhubung ke 1 switch.
 - Menambahkan perangkat NAT yang terhubung ke switch untuk menyediakan koneksi internet agar dapat melakukan cloning dari repository git.
2. Melakukan configure IP statis pada node masing - masing. IP dikonfigurasi dalam subnet NAT lengkap dengan gateway (192.168.122.1) dan DNS Nameserver (8.8.8.8) yang berfungsi untuk menghubungkan dengan koneksi internet.

3. Melakukan git clone dari repository yang telah diberikan pada setiap node dengan command git clone (untuk tugas ini dapat menggunakan <https://github.com/bwbrad05/Tugas-2-Sistem-Terdistribusi.git>)
4. Pada pc masing - masing, diarahkan menuju ke directory masing - masing node sesuai dengan pc yang digunakan (misal ds-computer-1 menggunakan directory node1 dan seterusnya).
5. Menjalankan logger pada directory logger dalam hostname ds-computer-4 dengan command ./run.bash untuk memantau aktivitas dari semua node berdasarkan Lamport clock, Vector Clock, dan physical time.

Code pada run.bash:

```
#!/bin/bash
```

```
python3 ./kv.py --logger --logger-tcp 9000 --numnodes 5
```

- --logger menandakan untuk menjalankan node sebagai logger.
 - --logger-tcp 9000 menandakan logger dijalankan dalam port TCP 9000.
 - --numnodes 5 menandakan banyaknya jumlah node yang terhubung adalah 5.
6. Jalankan node - node lain yang akan melakukan komunikasi melalui run.bash (node 1, node 2, node 3, node 4, dan node 5) dengan command yang sama yaitu ./run.bash

a. Node 1

Code pada run.bash:

```
#!/bin/bash
```

```
python3 ./kv.py --id 1 --tcp 8001 --udp 8101 --peers
192.168.122.11:8002=2,192.168.122.12:8003=3,192.168.122.1
4:8004=4,192.168.122.15:8005=5 --logger-addr
192.168.122.13:9000 --numnodes 5 --use-mutex 1
```

- --id 1 menandakan id yang dimiliki node adalah 1.
- --tcp 8001 menandakan port TCP untuk client requests dan replications dilakukan di 8001
- --udp 8101 menandakan port UDP yang digunakan untuk gossip protocol adalah 8101.
- --peers 192.168.122.11:8002=2,192.168.122.12:8003=3,192.168.122.14:8004=4,192.168.122.15:8005=5 digunakan untuk memberikan list node - node lain yang terhubung.
- --logger-addr 192.168.122.13.9000 menandakan semua event akan dikirim ke logger pada address tersebut dengan port 9000.
- --numnodes 5 menandakan banyaknya jumlah node yang terhubung adalah 5.
- --use-mutex 1 menandakan mutex diaktifkan untuk simulasi mutex (untuk membandingkan dengan simulasi tanpa mutex, dapat diubah menjadi --use-mutex 0).

Notes: Penjelasan untuk code run.bash pada node lain kurang lebih sama, hanya berbeda pada id, port TCP, port UDP, dengan daftar list node pada peers.

b. Node 2

Code:

```
#!/bin/bash
```

```
python3 ./kv.py --id 2 --tcp 8002 --udp 8102 --peers
192.168.122.10:8001=1,192.168.122.12:8003=3,192.168.122.1
4:8004=4,192.168.122.15:8005=5 --logger-addr
192.168.122.13:9000 --numnodes 5 --use-mutex 1
```

- --id 2 menandakan id yang dimiliki node adalah 2.
- --tcp 8002 menandakan port TCP untuk client requests dan replications dilakukan di 8002.
- --udp 8102 menandakan port UDP yang digunakan untuk gossip protocol adalah 8102.

c. Node 3

Code:

```
#!/bin/bash
```

```
python3 ./kv.py --id 3 --tcp 8003 --udp 8103 --peers
192.168.122.10:8001=1,192.168.122.11:8002=2,192.168.122.1
4:8004=4,192.168.122.15:8005=5 --logger-addr
192.168.122.13:9000 --numnodes 5 --use-mutex 1
```

- --id 3 menandakan id yang dimiliki node adalah 3.
- --tcp 8003 menandakan port TCP untuk client requests dan replications dilakukan di 8003.
- --udp 8103 menandakan port UDP yang digunakan untuk gossip protocol adalah 8103.

d. Node 4

Code:

```
#!/bin/bash
```

```
python3 ./kv.py --id 4 --tcp 8004 --udp 8104 --peers
192.168.122.10:8001=1,192.168.122.11:8002=2,192.168.122.1
2:8003=3,192.168.122.15:8005=5 --logger-addr
192.168.122.13:9000 --numnodes 5 --use-mutex 1
```

- --id 4 menandakan id yang dimiliki node adalah 4.
- --tcp 8004 menandakan port TCP untuk client requests dan replications dilakukan di 8004.

- --udp 8104 menandakan port UDP yang digunakan untuk gossip protocol adalah 8104.

e. Node 5

Code:

```
#!/bin/bash
```

```
python3 ./kv.py --id 5 --tcp 8005 --udp 8105 --peers
192.168.122.10:8001=1,192.168.122.11:8002=2,192.168.122.1
2:8003=3,192.168.122.14:8004=4 --logger-addr
192.168.122.13:9000 --numnodes 5 --use-mutex 1
```

- --id 5 menandakan id yang dimiliki node adalah 5.
- --tcp 8005 menandakan port TCP untuk client requests dan replications dilakukan di 8005.
- --udp 8105 menandakan port UDP yang digunakan untuk gossip protocol adalah 8105.

Penjelasan Komunikasi Antar Node

Protokol Transport

Protokol transport yang digunakan untuk komunikasi antar node adalah TCP dan UDP. Protokol TCP digunakan untuk aktivitas RPC (Remote Procedure Call) meliputi PUT, GET, REPL_PUT. Selain itu, digunakan juga untuk komunikasi mutex yaitu LOCK_REQ dan LOCK_REL, serta untuk mencatat komunikasi antar node dengan logger. Protokol TCP tersebut digunakan karena memiliki koneksi yang reliable dan memberikan urutan ordering yang terjamin. Di lain sisi, protokol UDP digunakan ketika melakukan Gossip membership protocol meliputi update heartbeat antar node dan juga failure detection. Protokol UDP cocok digunakan untuk aktivitas tersebut karena aktivitas - aktivitas tersebut tidak memerlukan pesan untuk reliable dan tidak mementingkan urutan ordering.

Protokol Message Exchange

1. Client-Server Commands
Protokol transport: TCP
 - a. PUT command
 - Format: PUT <key> <value>
 - Arah dari client menuju ke node.
 - Respons dari node berupa OK yang menandakan bahwa value berhasil disimpan.
 - b. GET command
 - Format: PUT <key> <value>
 - Arah dari client menuju ke node.
 - Respons dari node berupa <value> jika ada dan null jika tidak ada.

2. Replication Protocol
 - a. REPL_PUT command
 - Protokol transport: TCP
 - Format: REPL_PUT <key> <value> <lamport> <vector_json>
 - Arah dari node menuju ke node yang lainnya.
 - Respons berupa OK yang menandakan update telah diterima.
3. Logger Protocol
 - Protokol transport: TCP
 - a. STATE_UPDATE
 - Format: STATE_UPDATE <key> <value> <lamport>
 - Arah dari node menuju ke logger
 - Untuk mencatat perubahan state ketika logging.
 - Respons berupa LOGGED.
 - b. HEARTBEAT
 - Format: HEARTBEAT <id_node>
 - Arah dari node menuju ke logger.
 - Untuk memberi tahu bahwa node masih aktif.
 - Respons berupa OK.
 - c. SYNC_REQ
 - Format: SYNC_REQ <id_node>
 - Arah dari node menuju ke logger
 - Untuk meminta copy dari state saat node baru hidup
 - Respons berupa SYNC_DATA <json_state>.
4. Gossip Protocol
 - Digunakan untuk sinkronisasi ringan antar node
 - Protokol transport: UDP
 - Format: JSON
 - Arah dari node menuju ke node yang lainnya
5. Mutex Protocol
 - Protokol transport: UCP
 - a. LOCK_REQ command:
 - Format: LOCK_REQ <id_node>
 - Arah dari node menuju ke coordinator atau leader.
 - Respons berupa GRANTED jika shared resource tidak ada yang menggunakan atau QUEUED jika shared resource sedang digunakan.
 - b. LOCK_REL
 - Format: LOCK_REL <id_node>
 - Arah dari node menuju ke coordinator.
 - Respons berupa OK menandakan lock telah dibuka dan critical section telah selesai.

Penjelasan Simulasi Mutual Exclusion

Simulasi race condition untuk menguji penggunaan mutex dilakukan dengan cara run [test3.sh](#) pada folder testing. Pengujian race condition pada mutex dilakukan oleh command

```
python3 ./kvclient.py --nodes
```

```
192.168.122.10:8001,192.168.122.11:8002,192.168.122.12:8003,19
```

```
2.168.122.13:8004,192.168.122.14:8005 race "PUT color blue"
```

```
"PUT color green"
```

```
python3 ./kvclient.py --nodes
```

```
192.168.122.10:8001,192.168.122.11:8002,192.168.122.12:8003,19
```

```
2.168.122.13:8004,192.168.122.14:8005 race "PUT warna oranye"
```

```
"PUT warna hijau"
```

dengan pada race pertama adalah dilakukannya put color blue dan green diikuti dengan race kedua yang dilakukan put warna oranye dengan warna hijau.

a. Race Condition tanpa Mutex

```
[192.168.122.11:8002] PUT color black -> OK (5.84 ms)
[192.168.122.12:8003] PUT color magenta -> OK (6.76 ms)
[192.168.122.12:8003] PUT warna kuning -> OK (8.30 ms)
[192.168.122.12:8003] PUT warna kuning -> OK (7.12 ms)
[192.168.122.10:8001] PUT color blue -> OK (10.35 ms)
[192.168.122.11:8002] PUT color green -> OK (7.56 ms)
[192.168.122.10:8001] PUT warna oranye -> OK (8.20 ms)
[192.168.122.11:8002] PUT warna hijau -> OK (7.93 ms)
-----HASIL-----
[192.168.122.10:8001] GET color -> green (1.81 ms)
[192.168.122.11:8002] GET color -> blue (1.42 ms)
[192.168.122.12:8003] GET color -> blue (1.06 ms)
```

```
-----HASIL-----
[192.168.122.10:8001] GET warna -> hijau (4.38 ms)
[192.168.122.11:8002] GET warna -> oranye (1.49 ms)
[192.168.122.12:8003] GET warna -> hijau (1.46 ms)
```

Penjelasan:

Dapat dilihat dari hasil screenshot yang diberikan, jika --use-mutex 0 atau menandakan mutex sedang dimatikan, hasil value dari color ataupun warna yang dihasilkan tidak konsisten. Dari hasil value color masing - masing dapat dilihat bahwa ketika GET pertama menghasilkan green dan kedua terakhir menghasilkan blue. Di lain sisi, pada saat GET value dari warna juga menghasilkan value yang berbeda berupa hijau, oranye, kemudian hijau kembali. Hal tersebut membuktikan bahwa ketika mutex tidak diaktifkan, hasil PUT value akan mengalami race condition dengan yang melakukan write value terakhir akan menang karena shared resource tersebut dapat dilakukan akses secara bersamaan. Oleh karena itu, tanpa menggunakan mutex,

dapat disimpulkan bahwa hasil akhir value akan deterministik dan bergantung pada timing dan urutan dari replikasi.

b. Race Condition dengan Mutex

```
[192.168.122.11:8002] PUT color black -> OK (867.52 ms)
[192.168.122.12:8003] PUT color magenta -> OK (3360.42 ms)
[192.168.122.12:8003] PUT warna kuning -> OK (9.59 ms)
[192.168.122.12:8003] PUT warna kuning -> OK (442.31 ms)
[192.168.122.10:8001] PUT color blue -> OK (13.61 ms)
[192.168.122.11:8002] PUT color green -> OK (1650.85 ms)
[192.168.122.10:8001] PUT warna oranye -> OK (8.06 ms)
[192.168.122.11:8002] PUT warna hijau -> ERR: timed out (0.00 ms)
-----HASIL-----
[192.168.122.10:8001] GET color -> green (2.16 ms)
[192.168.122.11:8002] GET color -> green (0.91 ms)
[192.168.122.12:8003] GET color -> green (1.26 ms)
```

```
-----HASIL-----
[192.168.122.10:8001] GET warna -> oranye (1.56 ms)
[192.168.122.11:8002] GET warna -> oranye (1.36 ms)
[192.168.122.12:8003] GET warna -> oranye (2.64 ms)
```

Penjelasan:

Dari hasil screenshot di atas, dapat dilihat bahwa ketika `--use-mutex 1` atau mutex diaktifkan, hasil dari value color dan warna menjadi konsisten. Ketika mutex diaktifkan, hanya 1 node saja yang masuk ke dalam critical section dan dapat melakukan PUT command tanpa ada pengaruh atau race condition dengan node lain. Ketika ada node lain yang ingin mengakses resource tersebut ketika resource sedang digunakan, node tersebut akan dimasukkan ke dalam queue. Ketika shared resource sudah selesai digunakan oleh node yang sebelumnya, node yang pertama dalam queue tersebut akhirnya dapat mengakses resource tersebut dan juga masuk ke dalam critical section dan menutup akses shared resource secara sementara untuk node lain. Oleh karena itu, dapat disimpulkan dengan menggunakan mutex, data yang dihasilkan oleh masing - masing node akan konsisten dan sama.

Kesimpulan

Secara keseluruhan, keberadaan mutex sangat penting untuk mengontrol akses shared resource kepada banyak pengguna. Tanpa adanya mutex, terbukti bahwa akan terjadinya race condition dengan penulis terakhir yang akan mengganti value yang akhirnya akan menyebabkan hasil data menjadi tidak pasti dan deterministik yang pastinya tidak bagus untuk suatu sistem terdistribusi. Oleh karena itu, untuk seluruh sistem terdistribusi, diperlukan adanya protokol mutex agar dapat menghasilkan output data yang konsisten.